ComPDF Generation

Java SDK

开发者指南

Version 1.3.0

2026-10-19

Java 开发、邮件转换及打印配置

可点击目录与章节书签

目录

1 概述 4
1.1 ComPDF Generation SDK 4
1.2 主要功能 4
1.2.1 使用 HTML 生成 PDF 4
1.2.2 使用布局 API（Layout API）生成 PDF 4
1.2.3 邮件转换为 HTML 和 PDF 4

2 入门指南 5
2.1 系统要求 5
2.1.1 获取许可证密钥 5
2.1.2 系统要求 5
2.2 ComPDF Generation SDK 包结构 5
2.3 应用许可证密钥 6
2.3.1 复制许可证密钥 6
2.3.2 应用许可证密钥 6
2.4 怎样运行示例 6
2.5 怎样在 Java 应用中使用 ComPDF Generation SDK 7
2.6 示例 8
2.6.1 HTML 转 PDF 示例 8
2.6.2 使用布局 API 生成 PDF 示例 8
2.6.3 邮件与打印配置示例 8

3 功能指南 9
3.1 介绍 9
3.2 使用 HTML 生成 PDF 9
3.2.1 介绍 9
3.2.2 通过 HTML 模板生成 PDF 10
3.2.3 HTML 转换为 PDF 16
3.2.4 页面边距 17
3.2.5 页眉与页脚 17
3.3 使用布局 API 生成 PDF 19
3.3.1 介绍 19
3.3.2 核心概念 19
3.3.3 快速开始 20
3.3.4 文本和段落 21
3.3.5 表格 22
3.3.6 列表 23

3.3.7 图片 . . . 24
3.3.8 样式管理 . . . 25
3.3.9 盒模型 . . . 25
3.3.10 定位 . . . 26
3.3.11 背景 . . . 27
3.3.12 分隔线 . . . 27
3.3.13 字体管理 . . . 28
3.3.14 完整示例 . . . 29
3.3.15 最佳实践 . . . 32
3.3.16 常见问题 . . . 33
3.4 邮件转换为 HTML 和 PDF . . . 33
3.4.1 格式与初始化 . . . 33
3.4.2 邮件文件直接转换为 PDF . . . 34
3.4.3 分别导出 HTML 与生成 PDF . . . 34
3.4.4 选项与取消 . . . 35
3.4.5 流与内存输入 . . . 36
3.4.6 结果数据与状态码 . . . 38
3.4.7 生命周期与并发 . . . 39
3.4.8 邮件示例 . . . 39

4 技术支持 . . . 40
4.1 报告问题 . . . 40
4.2 联系信息 . . . 40

1 概述

ComPDF Generation SDK for Java 是一个功能强大的 PDF 生成库，提供多种方式生成高质量的 PDF 文档。无论是通过 HTML 转换还是直接通过代码构建，都能满足您的各种 PDF 生成需求。

1.1 ComPDF Generation SDK

ComPDF Generation SDK for Java 目前支持 Linux。

1.2 主要功能

PDF 生成（Generation）组件提供以下功能：

1.2.1 使用 HTML 生成 PDF

- **HTML 转换 PDF**：支持将 HTML 字符串、文件或流转换为 PDF 文档
- 模板渲染：支持使用 JSON 数据动态填充 HTML 模板，并渲染为 PDF 文档
- 自定义配置：可设置页面大小、页边距、基础 URI、字体及页眉页脚模板
- 字体管理：支持自定义字体，并可集成系统字体

1.2.2 使用布局 API（Layout API）生成 PDF

- 编程式文档构建：通过代码直接创建和控制 PDF 文档的每一个元素
- 丰富的元素类型：支持文本、段落、表格、列表、图片、分隔线等多种元素
- 灵活的布局控制：支持块级元素、行内元素和绝对定位，提供精确的布局控制
- 完整的样式系统：支持可重用样式、盒模型（边距、内边距、边框）、背景、字体等
- 流式布局：自动处理分页和内容重排，支持表格表头表尾在多页重复
- 树形文档结构：采用直观的层次结构组织文档内容，便于理解和维护

布局 API 特别适合需要精确控制文档布局、动态生成复杂结构文档，或需要与现有代码逻辑深度集成的场景。

1.2.3 邮件转换为 HTML 和 PDF

- **EML 与 MSG 输入**：支持从邮件文件、字节数组或流转换为 HTML 或 PDF。
- **HTML 包与附件**：导出选定的邮件正文、资源和可用附件。
- 转换控制：查询状态、诊断和正文来源，配置字节预算及协作式取消。

2 入门指南

2.1 系统要求

在开始集成前，请确保满足以下前提条件。

2.1.1 获取许可证密钥

ComPDF 提供两种许可证类型：30 天免费试用许可证和商业许可证。

如何获取试用许可证密钥

您可以直接在线注册，获取 ComPDF Generation SDK 的 30 天免费试用许可证：

1. 打开 ComPDF Portal 注册账号；已有账号可直接登录。
2. 选择 ComPDF Generation SDK 和 Linux 平台，按页面提示获取试用许可证。
3. 复制 License Key，并按照第 2.3 节的说明初始化 SDK。

通过在线注册即可获取试用许可证，无需联系销售。

如何获取商业许可证密钥

ComPDF Generation SDK 是商用
SDK，需要许可证才能授权开发者开发或发布其应用程序。请注意，即使拥有了许可证，也禁止将 ComPDF 的任何文件、示例代码或源代码给任何第三方。

如需获得 ComPDF Generation SDK 的商业许可证，或咨询报价，请随时联系我们的销售团队。

2.1.2 系统要求

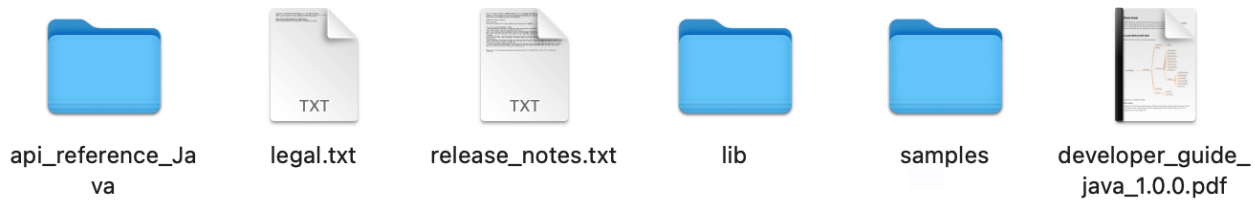
开发环境	**Java JDK 1.8 and higher**
**支持平台 **	**Linux (x86)**

2.2 ComPDF Generation SDK 包结构

ComPDF Generation SDK for Java 的 SDK 包包括以下文件：

- **"samples"** - Java 示例项目。
  - **"ConvertToPdf"** - HTML 转 PDF。
  - **"RenderTemplate"** - HTML 模板与 JSON 结合生成完整 HTML。
  - **"RenderAndConvert"** - 渲染模板并生成 PDF。
  - **"PrintOptionsTest"** - 页边距及页眉页脚配置。
  - **"FamilyHeaderFooterTest"** - 使用封面跳页及编号配置转换家庭 HTML 示例。
  - **"EmailConvertTest"** - 解析 EML/MSG，演示 HTML 包、文件 PDF 和流 PDF 输出。
  - **"TestFiles"** - 示例输入，包含 `email` 和 `family_html`。
  - **"lib"** - 示例依赖 `generation-1.3.0.jar`。
- **"generation-1.3.0.jar"** - SDK JAR，包含匹配的 Linux native 库。
- **"API Reference"** - Java API 参考，入口为 `index.html`。

- **"develop_guide_java_zh.pdf"** - 中文开发者指南。
- **"release_notes.txt"** - 版本更新记录。
- **"legal.txt"** - 法律和版权信息。



2.3 应用许可证密钥

如果您还没有 License Key，可先注册或登录 ComPDF Portal 在线获取，具体步骤见获取许可证密钥。

2.3.1 复制许可证密钥

从 ComPDF Portal 获取 ComPDF Generation SDK 许可证后，复制该许可证对应的完整 License Key，并按下方示例传给 `NativeComPDF.initialize()`。

如果获取的是 `XML` 许可证文件，请打开文件，仅复制 `<key>` 元素中的值。例如：

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<license version="1">
<platform>linux</platform>
<starttime>xxxxxxxx</starttime>
<endtime>xxxxxxxx</endtime>
<key>LICENSE_KEY</key>
</license>
```

使用其中的 `LICENSE_KEY` 值，不包含 XML 标签。请根据获取的许可证类型，选择对应的在线或离线认证方式。

2.3.2 应用许可证密钥

您可以通过以下方式进行认证：

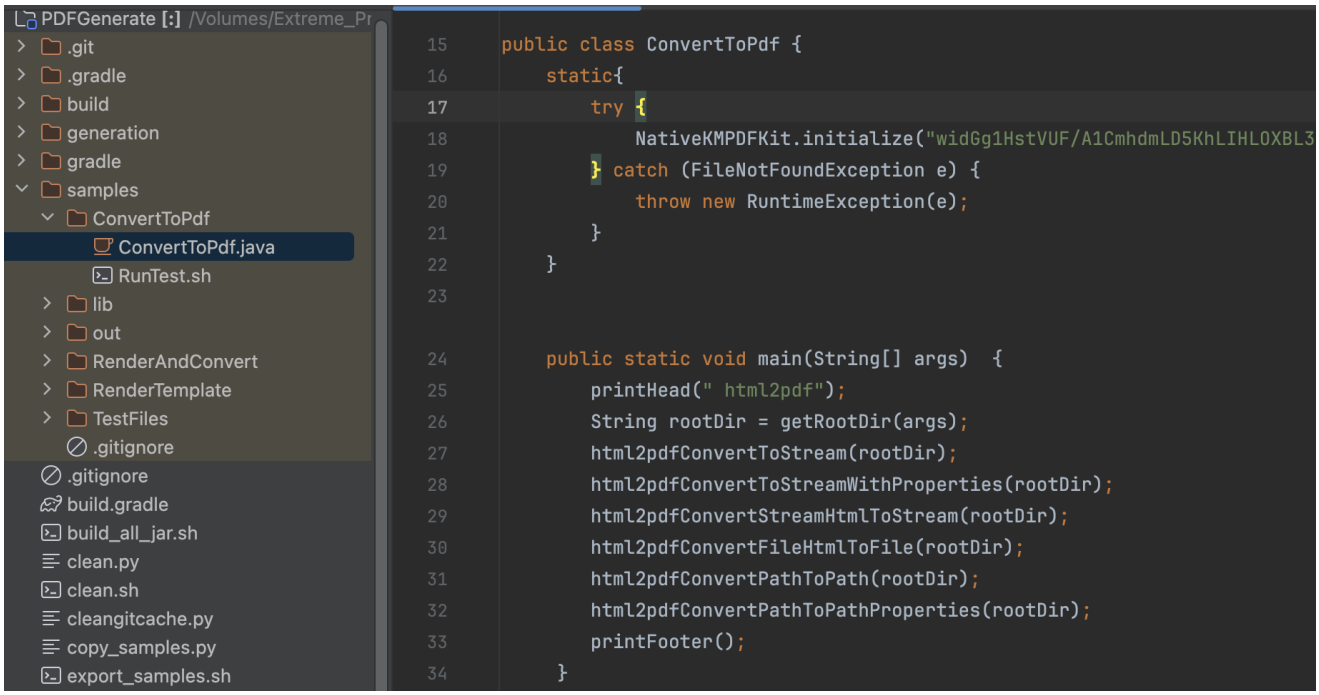
```
NativeComPDF.initialize("LICENSE_KEY", "your deviceId", "", Constants.LIB_DIR_NAME);
```

2.4 怎样运行示例

ComPDF Generation SDK for Java 为开发者提供了多个示例，以帮助他们学习如何在 Java 应用程序中调用该 SDK。您可以在 ***"samples"*** 文件夹中找到这些示例。在本指南中，我们以 ***"ConvertToPdf"*** 为例，展示如何在 Java 应用程序中运行它。

1. 在 IDE 中导入 ***"samples"*** 项目。
2. 进入 `samples/ConvertToPdf`，运行其中的 `RunTest.sh`：

```
./RunTest.sh
```

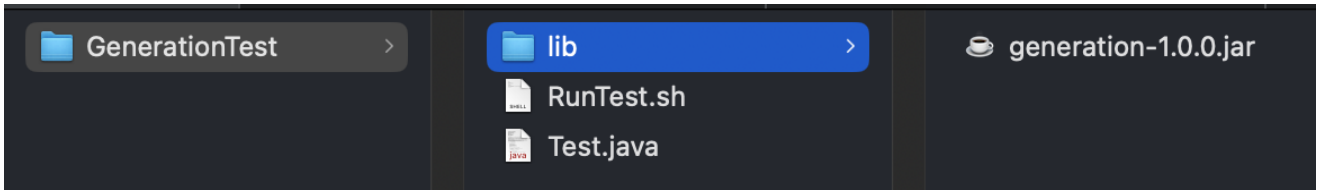


注意： 这是一个演示项目，展示了完整的 *ComPDF Generation SDK* 功能。功能可能根据您购买的许可证而有所不同。请检查您选择的功能在该演示项目中是否正常工作。输出文件将位于`/samples/out`。

2.5 怎样在 Java 应用中使用 ComPDF Generation SDK

本节将帮助您快速入门 ComPDF Generation SDK，以逐步指导您制作 Java 应用程序。

Java 工程结构：



1. 从 PDF Genenation SDK 中复制 generation-1.3.0.jar 到你的工程目录
2. 在 Java 工程目录中创建 RunTest.sh 文件：

```
#Test是要执行的类名，就是Java project中包含main方法的类。
TEST_NAME=Test
#SDK中提供的jar包名字
JAR_NAME=generation-1.3.0.jar
#Java project 根目录
ROOTDIR=$(pwd)
#生成库的jar包路径
LIB_DIR=$(pwd)/lib
#传入jar包路径，编译java文件
javac -cp $LIB_DIR/$JAR_NAME *.java
java -Djava.library.path=LIB_DIR -cp .:$LIB_DIR/$JAR_NAME $TEST_NAME $ROOTDIR
#删除中间生成的无用文件
rm -rf pdfgeneration
rm -rf *.class
```

3. 在终端中进入 Java 工程目录， 执行 `./RunTest.sh`。

2.6 示例

Samples 使用预设参数、文档和模块化的代码示例来调用 ComPDF Generation SDK 的每个功能的 API，无需 UI 交互或参数设置。不仅展示了 HTML 转 PDF 功能，还提供了使用布局 API 直接创建 PDF 文档的完整示例。

这些项目不仅展示了每个功能的最佳实践，还提供了详细的介绍。可以在输出目录中观察到每个功能对 PDF 文档的影响。借助 Samples，你可以快速学习如何使用你需要的功能，并将其应用到你的项目中。

2.6.1 HTML 转 PDF 示例

名称	描述
ConvertToPdf	HTML 转为 PDF。
RenderTemplate	HTML 模板与 JSON 结合生成完整的 HTML。
RenderAndConvert	将 HTML 模板与 JSON 数据结合，生成完整的 HTML，并将该完整 HTML 转换为 PDF 。

2.6.2 使用布局 API 生成 PDF 示例

名称	描述
CreateParagraph	创建具有各种文本格式选项的段落。
CreateTable	创建带有表头、表尾和样式化单元格的表格。
CreateList	创建具有多种样式的有序列表和无序列表。

2.6.3 邮件与打印配置示例

名称	描述
EmailConvertTest	解析 EML/MSG，导出离线 HTML 包，从文件或流生成 PDF。
PrintOptionsTest	设置与读取页边距和页眉页脚选项。
FamilyHeaderFooterTest	使用家庭 HTML 示例演示封面跳页及页码模板。

3 功能指南

如果您对概述部分提到的所有功能感兴趣，请仔细阅读我们的指南，以便快速将 PDF 生成功能添加到您的项目中。以下部分列出了一些示例，展示了如何使用我们的 Java API 在 Java 项目中使用 PDF 生成功能。如果您遵循我们提供的指南和示例，您将能够迅速使用我们的 Java API 将 PDF 生成功能集成到您的 Java 项目中。

3.1 介绍

使用 ComPDF Generation SDK for Java 库在您的应用程序中实现快速、大批量的 PDF 生成，我们提供以下不同的 PDF 生成方式，您可以结合您的实际业务场景选择不同的方法。

1. 通过 **HTML** 模板生成 **PDF**：只需创建一次 HTML 模板，再使用 JSON 数据填充 HTML 模板即可批量生成不同的 PDF，同时保留模板原有的页面布局；支持插入文本、图像以及动态表格，非常适合生成复杂、精美的 PDF，您可以使用这种方式生成复杂的报告、发票、信函和合同等。
2. **HTML** 转 **PDF**：可以将标准的 HTML 文件或 HTML 字符串转为 PDF 文件，适合将现有的 HTML 内容快速转换为 PDF 文档。
3. 通过布局 **API** 生成 **PDF**：通过代码直接创建和控制 PDF 文档的每一个元素，提供最精细的控制能力。这种方式允许您完全通过编程方式从零开始构建 PDF 文档，非常适合需要精确控制文档布局、动态生成复杂结构文档，或需要与现有代码逻辑深度集成的场景。布局 API 支持丰富的元素类型（文本、段落、表格、列表、图片等），提供完整的样式系统和灵活的布局控制。
4. 邮件转换为 **HTML** 或 **PDF**：解析 EML/MSG，查询选定的正文与附件，导出离线 HTML 包或生成邮件 PDF，详见第 3.4 节。

HTML 生成 PDF 主要能力：

1. 速度快，能够满足大批量生成 PDF 的需求
2. 完美还原模板布局
3. 丰富的 HTML 标签支持和 CSS 样式支持
4. 支持文本重排，保持完美的页面布局
5. 支持动态表格

布局 API 生成 PDF 主要能力：

1. 完全的编程控制：精确控制文档中每个元素的位置、大小和样式
2. 丰富的元素类型：支持文本、段落、表格、列表、图片、分隔线等多种元素
3. 灵活的布局系统：支持块级元素、行内元素和绝对定位
4. 完整的样式系统：支持可重用样式、盒模型、背景、字体等
5. 流式布局：自动处理分页和内容重排，支持表格表头表尾在多页重复
6. 树形文档结构：采用直观的层次结构组织文档内容

立即开始体验！

3.2 使用 HTML 生成 PDF

3.2.1 介绍

支持通过 HTML 模板动态生成 PDF 文件，或者通过标准的 HTML 文件或 HTML 字符串生成 PDF 文件。

使用通过 HTML 模板生成 PDF 文件功能，您需要准备以下内容：

- **模板：**以 HTML 的形式设计您的模板
- **替换数据：**提供 JSON 格式的填充数据

使用通过 HTML 文件或 HTML 字符串生成 PDF 文件功能，需要准备标准的 HTML 文件或 HTML 字符串。

在通过 HTML 模板生成 PDF 的过程中，ComPDF 将会将您提供的数据与模板合并，从而高效地生成 PDF 文件，在填充的过程中，您还可以使用条件语句、循环语句等，方便您实现更为灵活的填充规则；
剩下的工作就交给 ComPDF 来处理。

3.2.2 通过 HTML 模板生成 PDF

整体流程概述

通用 HTML 模板生成 PDF 功能需要一个 HTML 模板以及用来填充模板的结构化数据，就是 JSON 数据，从而生成一个标准的 HTML。再使用生成 PDF 功能相应的接口就可以将标准的 HTML 转成 PDF 文档。

1. 构建 **HTML** 模板

模板是一个 HTML 文件，还需要一个与模板匹配的 JSON 数据。 JSON 数据是一个正常的 JSON 文件。HTML 模板文件需要结合 JSON 数据以生成一个标准的 HTML。 结合 JSON 数据和 HTML 模版生成 HTML 的过程支持变量、if 语句、for 循环、四则运算、常用函数。

HTML 模板示例：

```
<!doctype html>
<html>
<body style="margin: 0; padding: 0;">
<table cellpadding="0" cellspacing="0" width="100%" style="border-collapse: collapse; margin: 0 auto
    ">
  <tr>
    <td style="padding-top: 40px;">
      <img
          src="{% if compdfkit %}images/logo.png{% else if compdfkit2 %}images/logo2.png{% else %}im
              ages/logo3.png{% endif %}"
          alt="PDF Technologies"
          width="203"
          height="50">
    </td>
    <td style="padding-top: 40px; text-align: right;">
      <img src="images/invoice.png" alt="invoice" width="132">
      <td style="border: 1px solid #8A8EA8; padding: 2px 10px;">{{billNo}}</td>
    </td>
  </tr>
</table>
<table cellpadding="0" cellspacing="0" width="100%" style="border-collapse: collapse;">
  <!-- {% set totalprice = 0 %} -->
  <!-- {% for item in items %} -->
  <tr style="height:64px;color:#333;">
    <td style="width:70px;height:70px;border-bottom: 2px solid #F3F3F3; padding-left: 10px; text-ali
        gn: left;">
      {{ item.index }}
    </td>
    <td style="width: 160px;border-bottom: 2px solid #F3F3F3; font-size: 13px; line-height: 15px; co
        lor: #121E3F;text-align:left;">
      <div>
        <p style="font-size: 13px; line-height: 15px; font-weight: bold; color: #121E3F; margin: 0;"
            >
          {{ item.product }}
        </p>
        <p style="font-size: 11px; line-height: 12px; padding-top: 4px; color: #5F6881; margin: 0;">
          {{ item.startDateAndEndDate }}
        </p>
```

```
      </div>
    </td>
    <td style="width: 130px;border-bottom: 2px solid #F3F3F3; font-size: 13px; line-height: 15px; co
        lor: #121E3F;text-align:left;">
      {{ item.paymentCycle }}
    </td>
    <td style="width: 100px;border-bottom: 2px solid #F3F3F3; font-size: 13px; line-height: 15px; co
        lor: #121E3F;text-align:left;">
      {{ item.gearLevel }}
    </td>
    <td style="width: 100px;text-align:left;border-bottom: 2px solid #F3F3F3; font-size: 13px; line-
        height: 15px; color: #121E3F;">
      $ {{ item.price }}
    </td>
  </tr>
  <!-- {% set totalprice = totalprice + item.price %} -->
  <!-- {% endfor %} -->
  <tr>
    <td style="width:70px;"></td>
    <td style="width:160px;"></td>
    <td colspan="2" style="width: 100px; text-align:right; font-weight: bold; font-size: 16px; line-
        height: 18px; color: #121E3F; padding-right: 72px;">Total:</td>
    <td style="width: 100px; text-align:left; font-weight: bold; font-size: 16px; line-height: 18px;
         color: #121E3F;">$ {{round(totalprice * (1 + tax * 0.01), 2)}}</td>
  </tr>
</table>
</body>
</html>
```

如上面示例中所示，HTML 模板中，存在一些非 HTML 标准的内容，比如： `{% if compdfkit %}`, `{{billNo}}`, `<!-- {% set totalprice = totalprice + item.price %} -->`, `<!-- {% for item in items %} -->` 等。这些非 HTML 标准的内容包括变量、if 语句、for 循环、四则运算、常用函数，这几种类型。

变量

```
<td style="border: 1px solid #8A8EA8; padding: 2px 10px;">{{billNo}}</td>
```

上面 `{{billNo}}` 就是一个变量，即变量需要 `{{ }}` 框住，对应 JSON 文件中名为`billNo` 的条目。

如果 JSON 文件中不存在相应名字，需要先定义。下面是定义变量的示例。

```
<!-- {% set totalprice = 0 %} -->
```

示例模板中，上面一行代码定义了一个 `totalprice`变量。和前面的 `billNo` 变量一样， `{{ }}`框住后使用：`{{totalprice}}`。

四则运算

```
<!-- {% set totalprice = totalprice + item.price %} -->
```

上面一行代码对`totalprice`进行计算，加减乘除都支持。

if 语句

```
<img
     src="{% if compdfkit %}images/logo.png{% else if compdfkit2 %}images/logo2.png{% else %}images/
         logo3.png{% endif %}"
     alt="PDF Technologies"
     width="203"
     height="50">
```

其中 `{% if compdfkit %} … {% else if compdfkit2 %}...{% else %}...{% endif %}` 是一个完整的 if 语句结构，`else if`和 `else` 可省略。`compdfkit`、`compdfkit2`对应着 JSON 文件中同名的条目。

for 循环

```
<!-- {% set totalprice = 0 %} -->
<!-- {% for item in items %} -->
                ·····.
      <!-- {% set totalprice = totalprice + item.price %} -->
<!-- {% endfor %} -->
```

`{% for item in items %}...{% endfor %}`是一个 for 循环的结构。其中`items`是 JSON 文件中的一个数组类型的数据。

常用函数

```
<td style="width: 100px; text-align:left; font-weight: bold; font-size: 16px; line-height: 18px; col
    or: #121E3F;">$ {{round(totalprice * (1 + tax * 0.01), 2)}}</td>
```

其中，`round`是函数，可以传入变量做为函数参数。`totalprice`、`tax`是变量。整个函数用`{{ }}`框住。

2. 准备 **JSON** 数据

准备标准格式的 JSON 数据，用来和 HTML 模板结合，将数据填充到模板中，以生成标准的 HTML。JSON 数据示例：

```
{
    "address": "456 Oak Avenue",
    "billNo": "2345678901",
    "company": "TechCorp Ltd.",
    "country": "Canada",
    "email": "alice.smith@techcorp.com",
    "firstName": "Alice",
    "gearLevel": "Pro",
    "compdfkit": false,
    "compdfkit2": true,
    "invoiceDate": "2025-01-15",
    "lastName": "Smith",
    "paymentCycle": "Yearly",
    "price": "1200",
    "province": "Ontario",
    "startDateAndEndDate": "2025-01-15 to 2025-02-15",
    "tax": 8,
    "zip": "K1A 0A6",
    "items": [
        {
            "index": 1,
            "product": "ComPDF Generation SDK",
            "startDateAndEndDate": "2025-01-15 to 2025-02-15",
            "paymentCycle": "Yearly",
            "gearLevel": "Pro",
            "price": 800
        },
        {
            "index": 2,
            "product": "ComPDF Generation SDK Support",
            "startDateAndEndDate": "2025-01-15 to 2025-02-15",
            "paymentCycle": "Monthly",
            "gearLevel": "Basic",
            "price": 400
        }
    ]
}
```

3. 填充模板，生成 **PDF**

JSON 文件与 HTML 模板结合转换为 PDF：

```
//HTML模板文件路径
String templatePath = "/template_1.html";
//JSON文件路径
String jsonPath = "/data1.json";
//生成的HTML文件路径
String outHtmlPath = "/outHtml.html";
//生成的PDF文件路径
String outPdfPath = "/outPdf.pdf";

//JSON 文件与 HTML模板结合生成 HTML
CHtmlConverter.CErrorCode id = CHtmlConverter.HtmlRenderTemplate(templatePath, jsonPath, outHtmlPath
    );
// HTML 转为 PDF
id = CHtmlConverter.ConvertToPdf(outHtmlPath, outPdfPath);
```

这个示例中，`HtmlRenderTemplate` 和 `ConvertToPdf` 这两步可以由 `RenderAndConvertToPdf`这一个接口替代。

```
//HTML模板文件路径
String templatePath = "/template_1.html";
//JSON文件路径
String jsonPath = "/data1.json";
//生成的PDF文件路径
String outPdfPath = "/outPdf.pdf";
CHtmlConverter.CErrorCode id = CHtmlConverter.RenderAndConvertToPdf(templatePath, jsonPath, outPdfPa
    th);
```

4. 生成的 PDF 示例



ComPDF
Generation SDK

INVOICE

Invoice Number: 2345678901
Invoice Date: 2025-01-15
Page: 1

Bill To:

Name	Smith Alice
Email	alice.smith@techcorp.com
Company	TechCorp Ltd.
Country	Canada
Province	Ontario
ZIP Code	K1A 0A6
Address	456 Oak Avenue

Bill From:

PDF Technologies, Inc.

49F-0708, Changsha Shimao World Financial Center, No. 393 Jianxiang Road, Furong District, Changsha City, China

support@compdf.com

Invoice Details:

No.	Product	Payment Cycle	Plan	Price
1	ComPDF Generation SDK 2025-01-15 to 2025-02-15	Yearly	Pro	$ 800
2	ComPDF Generation SDK Support 2025-01-15 to 2025-02-15	Monthly	Basic	$ 400

Tax:	8%
Total Excluding Tax:	$ 1200
Total:	$ 1296.00

This is a computer-generated invoice. No signature is required.

Paying by Bank Transfer:

Bank: Agricultural Bank of China, Changsha Furong Sub-branch
Account Number: 18099914040001269
Account Holder: PDF TECHNOLOGIES, INC.
SWIFT Code: ABOCCNBJ180

ComPDF Generation SDK | support@compdf.com

配置属性

当转换 HTML 为 PDF 时，可以使用一些属性，像是 base uri、page size、font。

```
//HTML模板文件路径
String templatePath = "/test/template_1.html";
//JSON文件路径
String jsonPath = "/test/data1.json";
//生成的HTML文件路径
String outHtmlPath = "/test/outHtml.html";
//生成的PDF文件路径
String outPdfPath = "/test/outPdf.pdf";

//将 JSON 文件与HTML模板结合生成 HTML
CHtmlConverter.CErrorCode id = CHtmlConverter.HtmlRenderTemplate(templatePath, jsonPath, outHtmlPath
    );

//当换时使用属性
CConverterProperties properties = new CConverterProperties();
//设置页size
```

```
CSize size = new CSize(400, 600);
properties.setPageSize(size);
//设置字体目录
CFontProvider fontProvider = new CFontProvider();
fontProvider.addDirectory("/test/fontDir/");
properties.setFontProvider(fontProvider);

// 使用属性，将HTML 转为 PDF
id = CHtmlConverter.ConvertToPdf(outHtmlPath, outPdfPath, properties);
```

这个示例使用了 `fontProvider.addDirectory("/test/fontDir/")`。应该提供一些字体文件，并放置到 `/fontDir/` 目录中，这样 HTML 文件中的文本就能使用这些提供的字体。 下面是部分 HTML 示例代码，其中 span 内的文本使用了 Sarabun 字体。Sarabun 是提供的字体，相应的字体文件应放置在正确的目录中。

```
<style data-dito-stylesheet-name="setFonts">@font-face {
    font-family: "Sarabun";
}
@font-face {
    font-family: "Sarabun";
    font-style: italic;
}
@font-face {
    font-family: "SarabunThin";
}
@font-face {
    font-family: "SarabunsemiBold";
}</style>

<div style="margin-left:0in;margin-top:0pt;margin-bottom:0pt">
  <span style="font-family:Sarabun">
    <span style="color:black">
      <span style="font-size:26px">(PEA GECC Internal Audit)</span>
    </span>
  </span>
</div>
```

在转换过程中，HTML 模板和 JSON 可以是内容字符串、InputStream、文件对象或文件路径。生成的 PDF 可以是 OutputStream、文件对象或文件路径。 前面的示例使用了文件路径，而以下是使用其他类型的示例。

使用输入流与输出流：

```
String templatePath = "/test/template_1.html";
String jsonPath = "/test/data1.json";
String outPdfPath = "/test/outPdf.pdf";
try {
    InputStream htmlIn = new FileInputStream(templatePath);
    InputStream jsonIn = new FileInputStream(jsonPath);
    OutputStream pdfOut = new FileOutputStream(outPdfPath);
    CHtmlConverter.CErrorCode error = CHtmlConverter.RenderAndConvertToPdf(htmlIn, jsonIn, pdfOut);

} catch (Exception e) {

}
```

使用内容字符串：

```
String getFileContent(String path) {
        StringBuilder content = new StringBuilder();
        try (BufferedReader br = new BufferedReader(new InputStreamReader(new FileInputStream(path))
            )) {
            String line;
            while ((line = br.readLine()) != null) {
                content.append(line).append(System.lineSeparator());
            }
            return content.toString();
        } catch (IOException e) {
```

```
            return "";
        }
    }

String templatePath = "/test/template_1.html";
String jsonPath = "/test/data1.json";
String outPdfPath = "/test/outPdf.pdf";
try {
    String templateStr = getFileContent(templatePath);
    String jsonStr = getFileContent(jsonPath);
    OutputStream pdfOut = new FileOutputStream(outPdfPath);
    CHtmlConverter.CErrorCode error = CHtmlConverter.RenderAndConvertToPdf(templateStr, jsonStr, pdf
        Out);
} catch (Exception e) {

}
```

使用文件：

```
String templatePath = "/test/template_1.html";
String jsonPath = "/test/data1.json";
String outPdfPath = "/test/outPdf.pdf";
try {
    File templateFile = new File(templatePath);
    File jsonFile = new File(jsonPath);
    File pdfFile = new File(outPdfPath);
    CHtmlConverter.CErrorCode error = CHtmlConverter.RenderAndConvertToPdf(templateFile, jsonFile, p
        dfFile);
} catch (Exception e) {

}
```

3.2.3 HTML 转换为 PDF

这种生成 PDF 的方式，不需要模板和 JSON 数据，只需一个标准的 HTML 文件，或者标准的 HTML 字符串。

```
 String htmlPath = "/test/convert.html";
 String baseUrl = "/test/";
 String outputPdf = "/test/convert.pdf";
 int pageWidth = 842;
 int pageHeight = 596;
 CConverterProperties properties = new CConverterProperties();
 CSize size = new CSize(pageWidth, pageHeight);
 properties.setPageSize(size);
 properties.setBaseUri(baseUrl);
 CFontProvider fontProvider = new CFontProvider();
 properties.setFontProvider(fontProvider);
 CHtmlConverter.CErrorCode id = CHtmlConverter.ConvertToPdf(htmlPath, outputPdf, properties);
```

HTML 可以是内容字符串、InputStream、文件对象或文件路径。生成的 PDF 可以是 OutputStream、文件对象或文件路径。 前面的示例使用了文件路径，而以下是使用其他类型的示例。

使用输入流与输出流：

```
String htmlPath = "/test/convert_1.html";
String outPdfPath = "/test/outPdf.pdf";
try {
    InputStream htmlIn = new FileInputStream(htmlPath);
    OutputStream pdfOut = new FileOutputStream(outPdfPath);
    CHtmlConverter.CErrorCode error = CHtmlConverter.ConvertToPdf(htmlIn, pdfOut);
} catch (Exception e) {

}
```

使用内容字符串：

```
String htmlPath = "/test/convert_1.html";
String outPdfPath = "/test/outPdf.pdf";
try {
    String htmlStr = getFileContent(htmlPath);
    OutputStream pdfOut = new FileOutputStream(outPdfPath);
    CHtmlConverter.CErrorCode error = CHtmlConverter.ConvertToPdf(htmlStr, pdfOut);
} catch (Exception e) {

}
```

使用文件：

```
String htmlPath = "/test/convert_1.html";
String outPdfPath = "/test/outPdf.pdf";
try {
    File htmlFile = new File(htmlPath);
    File pdfFile = new File(outPdfPath);
    CHtmlConverter.CErrorCode error = CHtmlConverter.ConvertToPdf(htmlFile, pdfFile);
} catch (Exception e) {

}
```

3.2.4 页面边距

1.3.0 版本在 `CConverterProperties` 中新增 `setPageMargins(top, right, bottom, left)` 和 `getPageMargins()`。页边距单位为 **CSS** 像素（**CSS px**），对每页生效；`setPageSize()` 的页面宽高单位为点（**point**）。正文 CSS `@page` 规则中显式指定的页边距优先于属性中的设置。

```
import CPDFGeneration.CConverterProperties;
import CPDFGeneration.CHtmlConverter;

CConverterProperties properties = new CConverterProperties();
try {
    properties.setPageMargins(48, 40, 48, 40);
    int[] margins = properties.getPageMargins();
    System.out.println("Top margin: " + margins[0] + " CSS px");
    CHtmlConverter.CErrorCode status = CHtmlConverter.ConvertToPdf(
            "input.html", "margins.pdf", properties);
    if (status != CHtmlConverter.CErrorCode.HTML_CONVERTER_SUCCESS) {
        throw new IllegalStateException("HTML conversion failed: " + status);
    }
} finally {
    properties.release();
}
```

四个边距必须为非负数。`getPageMargins()` 返回一个新数组，顺序为上、右、下、左；修改该数组不会更新属性。新建的 `CConverterProperties` 默认四个边距均为 0。邮件转换在未传入属性对象时使用独立的默认边距，详见第 3.4 节。

转换期间应保持属性对象及其字体提供者有效；使用结束后，按上例释放属性对象。

3.2.5 页眉与页脚

通过 `CHtmlHeaderFooterOptions` 和 `setHeaderFooterOptions()`，可在每页上、下边距区域渲染独立的 HTML 模板。将 `displayHeaderFooter` 设为 `true`，并为模板预留足够的上下边距。模板宽度或高度超出可用区域时，转换会失败；应增加边距或缩小模板。

下面的示例将第一个实际页面作为封面：隐藏封面的页眉页脚，并从第二个实际页面开始编号为 1。

```
import CPDFGeneration.CConverterProperties;
import CPDFGeneration.CHtmlConverter;
import CPDFGeneration.CHtmlHeaderFooterOptions;

CConverterProperties properties = new CConverterProperties();
```

```
try {
    properties.setPageMargins(48, 40, 48, 40);
    CHtmlHeaderFooterOptions options = new CHtmlHeaderFooterOptions();
    options.displayHeaderFooter = true;
    options.headerTemplate = "<div style='font-size:10px'>"
            + "<span class='title'></span></div>";
    options.footerTemplate = "<div style='font-size:10px;text-align:right'>"
            + "<span class='date'></span> | "
            + "<span class='pageNumber'></span>/<span class='totalPages'></span></div>";
    options.title = "Report";
    options.skipLeadingTemplatePages = 1;
    options.skipLeadingNumberedPages = 1;
    properties.setHeaderFooterOptions(options);

    CHtmlHeaderFooterOptions saved = properties.getHeaderFooterOptions();
    System.out.println("Headers/footers enabled: " + saved.displayHeaderFooter);
    CHtmlConverter.CErrorCode status = CHtmlConverter.ConvertToPdf(
            "input.html", "header-footer.pdf", properties);
    if (status != CHtmlConverter.CErrorCode.HTML_CONVERTER_SUCCESS) {
        throw new IllegalStateException("HTML conversion failed: " + status);
    }
} finally {
    properties.release();
}
```

字段	默认值	说明
`displayHeaderFooter`	`false`	是否启用页眉页脚。
`headerTemplate`	空字符串	页眉 HTML；空字符串使用默认页眉模板。
`footerTemplate`	空字符串	页脚 HTML；空字符串使用默认页脚模板。
`baseUri`	空字符串	模板资源的基础 URI；为空时使用正文资源目录。
`title`	空字符串	显示的标题；为空时使用正文的 HTML 标题。
`url`	空字符串	显示的 URL；转换没有来源 URL 的 HTML 时，应由调用方提供。
`date`	空字符串	显示的日期；为空时使用本次转换的本地日期和时间。
`skipLeadingTemplatePages`	`0`	隐藏前 N 个实际页面的模板，不改变编号。
`skipLeadingNumberedPages`	`0`	前 N 个实际页面不计入编号且不显示模板，后续页面从 1 开始编号。

两个跳页数必须为非负数。同时设置时，前 `max(skipLeadingTemplatePages, skipLeadingNumberedPages)` 个实际页面不显示模板；编号只由 `skipLeadingNumberedPages` 决定。

模板支持通过 CSS 类名 `date`、`title`、`url`、`pageNumber` 和 `totalPages` 填充动态内容，例如 `<span class='pageNumber'></span>`。页码还支持文本占位符 `{{pageNumber}}` 和 `{{totalPages}}`。`totalPages` 不包含 `skipLeadingNumberedPages` 跳过的页面。

模板为空字符串时使用内置模板。需要明确留空某一项时，使用 `<div></div>`。模板支持的 HTML/CSS 以渲染器能力为准，不执行 JavaScript。

`setHeaderFooterOptions()` 会复制配置；之后修改传入对象不会改变属性。`getHeaderFooterOptions()` 返回独立副本，修改副本后需再次调用 setter 才能生效。邮件转 PDF 时，也可以通过 `CConverterProperties` 使用这些设置。

3.3 使用布局 API 生成 PDF

3.3.1 介绍

ComPDF Generation SDK 提供了一套强大的布局 API（Layout API），允许您通过代码直接创建和控制 PDF 文档的每一个元素。与 HTML 转 PDF 方式不同，布局 API
提供了更精细的控制能力，您可以完全通过编程方式从零开始构建 PDF 文档。

主要优势：

- 完全的编程控制：通过代码精确控制文档中每个元素的位置、大小和样式
- 灵活的布局系统：支持块级元素、行内元素和绝对定位
- 丰富的元素类型：文本、段落、表格、列表、图片、分隔线等
- 样式管理：支持可重用样式，实现一致的文档外观
- 盒模型支持：完整支持边距（margin）、内边距（padding）、边框（border）等
- 流式布局：自动处理分页和内容重排

适用场景：

- 需要精确控制文档布局的场景
- 动态生成结构复杂的文档
- 需要复杂的条件逻辑来决定文档内容
- 需要与现有代码逻辑深度集成

3.3.2 核心概念

文档结构

布局 API 采用树形结构组织文档内容：

```
CDocument（文档）
  ├─ CPage（页面）
  │   ├─ CDiv（容器）
  │   │   ├─ CParagraph（段落）
  │   │   ├─ CTable（表格）
  │   │   ├─ CList（列表）
  │   │   ├─ CImage（图片）
  │   │   ├─ CText（文本）
  │   │   └─ CLineSeparator（分隔线）
  │   └─ ...（其他元素）
  └─ CStyle（样式）
```

元素类型

1. 容器元素

- **CDocument**：PDF 文档的根容器，可包含多个页面
- **CPage**：表示 PDF 的一个页面，可包含各种布局元素
- **CDiv**：通用容器元素，类似于 HTML 中的 div，用于组织内容
- **CParagraph**：段落容器，支持行间距和首行缩进
- **CTable**：表格元素，支持单元格合并、表头、表尾
- **CCell**：表格单元格
- **CList**：列表容器，支持有序和无序列表

- **CListItem**：列表项

2. 内容元素

- **CText**：文本元素，可设置字体、颜色、大小等
- **CImage**：图片元素，支持多种缩放模式

3. 其他元素

- **CLineSeparator**：水平分隔线
- **CStyle**：可重用的样式定义

单位系统

布局 API 使用 **CUnitValue** 结构来表示尺寸值，支持两种单位：

```
// 点(Point)单位 - 1 点 = 1/72 英寸
CUnitValue pointValue = CUnitValue.CreatePointValue(100f);

// 百分比单位
CUnitValue percentValue = CUnitValue.CreatePercentValue(50f);
```

颜色系统

使用 **CColor** 结构定义 RGBA 颜色：

```
// 使用 RGBA 值创建颜色
CColor color = new CColor(255, 0, 0, 255); // 红色,完全不透明

// 使用预定义颜色
CColor black = CColor.BLACK;
CColor white = CColor.WHITE;
CColor red = CColor.RED;
```

3.3.3 快速开始

创建第一个 **PDF** 文档

```
import CPDFGeneration.layout.*;

public class Example {
    public static void main(String[] args) {
        // 创建文档
        CDocument document = new CDocument();
        // 创建页面 (A4 尺寸: 595x842 点)
        CPage page = document.createPage(595, 842);
        // 创建 div 容器
        CDiv div = page.createDivElement();
        // 添加文本
        CText text = div.createTextElement("Hello, PDF Generation!");
        text.setFontSize(24);
        text.setFontColor(CColor.BLACK);
        // 保存到文件
        document.saveToFile("output.pdf");
    }
}
```

3.3.4 文本和段落

创建文本

```
CDiv div = document.createPage(800, 600).createDivElement();
// 在 div 中创建文本
CText text = div.createTextElement("这是一段文本");

// 设置字体属性
text.setFontFamily("Arial");
text.setFontSize(14);
text.setFontColor(new CColor(0, 0, 0, 255));
text.setFontWeight(CFontWeight.BOLD);
text.setFontStyle(CFontStyle.ITALIC);

// 设置文本装饰
text.setUnderline(CColor.BLACK);

// 设置字符和单词间距
text.setCharacterSpacing(0.5f);
text.setWordSpacing(2.0f);
```

创建段落

```
CDiv div = document.createPage(800, 600).createDivElement();
// 创建空段落
CParagraph paragraph0 = div.createParagraphElement();

//或创建带初始文本的段落
CParagraph paragraph = div.createParagraphElement("段落文本内容");

// 在段落中添加文本
CText text1 = paragraph.createTextElement("第一部分文本");
CText text2 = paragraph.createTextElement("第二部分文本");
text2.setFontWeight(CFontWeight.BOLD);

// 设置行间距
paragraph.setFixedLeading(20f); // 固定行间距
// 或
paragraph.setMultipliedLeading(1.5f); // 相对行间距(1.5 倍)

//设置首行缩进
paragraph.setFirstLineIndent(32f);

//设置文本对齐
paragraph.setTextAlignment(CTextAlignment.JUSTIFIED);
```

文本对齐方式

```
element.setTextAlignment(CTextAlignment.Left);    // 左对齐
element.setTextAlignment(CTextAlignment.Center);  // 居中对齐
element.setTextAlignment(CTextAlignment.Right);   // 右对齐
element.setTextAlignment(CTextAlignment.Justify); // 两端对齐
```

3.3.5 表格

创建基本表格

```
// 方式 1: 创建指定列数的表格(等宽列)
CTable table = div.createTableElement(3);

// 方式 2: 创建指定列宽的表格(点单位)
float[] columnWidths = new float[] { 100, 150, 100 };
CTable table1 = div.createTableElement(columnWidths);

// 方式 3: 创建指定列宽的表格(支持百分比)
CUnitValue[] columnWidths1 = new CUnitValue[] {
CUnitValue.CreatePercentValue(30),
CUnitValue.CreatePercentValue(40),
CUnitValue.CreatePercentValue(30)
};
CTable table2 = div.createTableElement(columnWidths1);

// 使用全部可用宽度
table2.useAllAvailableWidth();
```

添加表格内容

```
// 开始新行
table.startNewRow();

// 创建单元格
CCell cell1 = table.createCell(1, 1);
cell1.createTextElement("单元格 1");


CCell cell2 = table.createCell(1,1);
cell2.createTextElement("单元格 2");

// 创建跨行跨列的单元格
CCell cell3 = table.createCell(2, 2); // rowspan, colspan
cell3.createTextElement("跨 2 行 2 列");
```

表格表头和表尾

```
// 创建表头
table.startNewHeaderRow();
CCell headerCell1 = table.createHeaderCell(1,1);
headerCell1.createTextElement("列 1");
CCell headerCell2 = table.createHeaderCell(1,1);
headerCell2.createTextElement("列 2");

// 创建表尾
table.startNewFooterRow();
CCell footerCell1 = table.createFooterCell(1,1);
footerCell1.createTextElement("合计");
CCell footerCell2 = table.createFooterCell(1,1);
footerCell2.createTextElement("1000");

// 创建表格标题
CDiv caption = table.createCaption();
caption.createTextElement("销售数据表");
```

设置表格样式

```
// 设置边框折叠模式
table.setBorderCollapse(CBorderCollapseType.COLLAPSE);

// 设置边框间距(仅在 Separate 模式下有效)
table.setBorderCollapse(CBorderCollapseType.SEPARATE);
table.setHorizontalBorderSpacing(5f);
table.setVerticalBorderSpacing(5f);

// 设置单元格边框
CBorder border = new CBorder(CUnitValue.CreatePointValue(1),CBorder.CBorderType.SOLID, CColor.BLACK)
    ;
cell.setBorder(border);


// 设置单元格背景色
cell.setBackgroundColor(new CColor(240, 240, 240, 255));

// 设置单元格对齐
cell.setTextAlignment(CTextAlignment.CENTER);
cell.setVerticalAlignment(CVerticalAlignment.MIDDLE);
```

3.3.6 列表

无序列表

```
// 创建无序列表
CList list = div.createListElement();

// 设置列表符号
list.setListSymbol(CListSymbolType.DISC);    // 实心圆点
// list.setListSymbol(CListSymbolType.Circle); // 空心圆圈
// list.setListSymbol(CListSymbolType.Square); // 方块

// 或使用自定义文本符号
list.setListSymbol("→");

// 添加列表项
CListItem item1 = list.createListItemElement("第一项");
CListItem item2 = list.createListItemElement("第二项");
CListItem item3 = list.createListItemElement("第三项");
```

有序列表

```
// 创建有序列表
CList list = div.createList();

// 设置编号类型
list.setListSymbol(CListNumberType.DECIMAL);      // 1, 2, 3, ...
// list.setListSymbol(CListNumberType.ENGLISH_LOWER); // a, b, c, ...
// list.setListSymbol(CListNumberType.ENGLISH_UPPER); // A, B, C, ...
// list.setListSymbol(CListNumberType.ROMAN_LOWER); // i, ii, iii, ...
// list.setListSymbol(CListNumberType.ROMAN_UPPER); // I, II, III, ...

// 设置起始编号
list.setItemStartIndex(5); // 从 5 开始编号

// 添加列表项
list.createListItemElement("第一项");
list.createListItemElement("第二项");
```

嵌套列表

```
CList outerList = div.createListElement();
outerList.setListSymbol(CListNumberType.DECIMAL);

CListItem item1 = outerList.createListItemElement("主项 1");

// 在列表项中创建子列表
CList innerList = item1.createListElement();
innerList.setListSymbol(CListSymbolType.CIRCLE);
innerList.createListItemElement("子项 1.1");
innerList.createListItemElement("子项 1.2");

CListItem item2 = outerList.createListItemElement("主项 2");
```

3.3.7 图片

添加图片

```
// 从文件路径创建图片
CImage image = div.createImageElement("path/to/image.png");

// 设置图片尺寸(点单位)
image.setWidth(300);
image.setHeight(200);

// 或使用百分比
image.setWidth(CUnitValue.CreatePercentValue(50));
```

图片缩放

```
// 按比例缩放
image.scale(0.5f, 0.5f); // 缩放到 50%

// 适应指定尺寸(保持宽高比)
image.scaleToFit(400, 300);

// 缩放到精确尺寸(可能变形)
image.scaleAbsolute(400, 300);

// 自动缩放
image.setAutoScale(true);
image.setAutoScaleWidth(true);
image.setAutoScaleHeight(true);
```

图片定位

```
// 绝对定位
image.setAbsolutePosition(100, 100);

// 设置边距
image.setMargin(10);
image.setMargins(10, 20, 10, 20); // 上、右、下、左

// 设置内边距
image.setPadding(5);
```

3.3.8 样式管理

创建和使用样式

```
// 创建样式
CStyle headingStyle = document.createStyle();

// 设置样式属性(支持方法链)
headingStyle
    .setFontSize(18)
    .setFontWeight(CFontWeight.BOLD);

headingStyle.setMarginBottom(10)
    .setTextAlignment(CTextAlignment.CENTER);

// 应用样式到元素
CText heading = div.createTextElement("标题文本");
heading.addStyle(headingStyle);

// 元素也可以添加多个样式
CStyle colorStyle = document.createStyle();
colorStyle.setFontColor(CColor.RED);
heading.addStyle(colorStyle);
```

3.3.9 盒模型

边距

```
// 设置统一边距
div.setMargin(20);

// 设置各边边距
div.setMargins(10, 20, 10, 20); // 上、右、下、左

// 单独设置某一边
div.setMarginTop(10);
div.setMarginRight(20);
div.setMarginBottom(10);
div.setMarginLeft(20);
```

内边距

```
// 设置统一内边距
div.setPadding(15);

// 设置各边内边距
div.setPaddings(10, 15, 10, 15);

// 单独设置
div.setPaddingTop(10);
div.setPaddingRight(15);
div.setPaddingBottom(10);
div.setPaddingLeft(15);
```

边框（Border）

```
// 创建边框
 border.setType(CBorder.CBorderType.SOLID);  // Solid, Dashed, Dotted, Double
 border.setColor(new CColor(0, 0, 0, 255));

 // 应用到所有边
 div.setBorder(border);

 // 或分别设置各边
 div.setBorderTop(border);
 div.setBorderRight(border);
 div.setBorderBottom(border);
 div.setBorderLeft(border);
```

圆角边框

```
// 创建统一圆角
CBorderRadius radius = new CBorderRadius(10f);
div.setBorderRadius(radius);

//创建椭圆形圆角
CBorderRadius radius2 = new CBorderRadius(10f, 5f);
div.setBorderRadius(radius2);

//分别设置各个角
CBorderRadius topLeft = new CBorderRadius(10f);
div.setBorderTopLeftRadius(topLeft);
```

宽度和高度

```
// 设置固定尺寸
div.setWidth(400);
div.setHeight(300);

// 使用百分比
div.setWidth(CUnitValue.CreatePercentValue(50));

// 设置最大/最小尺寸
div.setMaxWidth(600);
div.setMinWidth(200);
div.setMaxHeight(400);
div.setMinHeight(100);
```

3.3.10 定位

相对定位（默认）

元素按照正常的文档流进行排列，这是默认行为。

```
CPage page = document.createPage(800, 600);
CDiv div1 = page.createDivElement();
div1.createTextElement("第一个 div");

CDiv div2 = page.createDivElement();
div2.createTextElement("第二个 div"); // 会出现在 div1 下方
```

绝对定位

```
// 创建绝对定位的元素
CDiv div = page.createDivElement();
div.setAbsolutePosition(
    100,    // left: 距离页面左边缘 100 点
    200,    // bottom: 距离页面底边缘 200 点
    300     // width: 宽度 300 点
);

// 或使用百分比宽度
div.setAbsolutePosition(
    100,
    200,
    CUnitValue.CreatePercentValue(50)
);
```

3.3.11 背景

设置背景颜色

```
div.setBackgroundColor(new CColor(240, 240, 240, 255));
```

设置背景图片

```
// 设置背景图片
CBackgroundImage bgImage = new CBackgroundImage();
 // 设置单个图片源
 bgImage.setSource("path/to/background.png");

 ArrayList<String> sources = new ArrayList();
 sources.add("path/to/image1.png");
 sources.add("path/to/image2.png");
 // 或设置多个图片源
 bgImage.setSources(sources);

 // 应用背景图片
 div.setBackgroundImage(bgImage);
```

3.3.12 分隔线

```
// 创建分隔线
CLineSeparator separator = div.createLineSeparatorElement();

//设置线条样式
separator.setLineWidth(2f);
separator.setLineType(CLineSeparator.CPDFLineType.SOLID); // Solid, Dashed, Dotted
separator.setLineColor(new CColor(200, 200, 200, 255));

//设置分隔线宽度(默认使用全部可用宽度)
separator.setWidth(CUnitValue.CreatePercentValue(80));
```

3.3.13 字体管理

```
// 创建字体提供器
CFontProvider fontProvider = new CFontProvider();

//添加字体目录
fontProvider.addDirectory("path/to/fonts");

//添加系统字体
fontProvider.addSystemFonts();

//将字体提供器应用到文档
document.setFontProvider(fontProvider);

//在元素中使用字体
div.setFontFamily("Arial");
```

3.3.14 完整示例

创建发票文档

```
import CPDFGeneration.layout.*;

public class InvoiceExample {
    public static void generateInvoice() {
        // 创建文档
        CDocument document = new CDocument();

            // 创建 A4 页面
            CPage page = document.createPage(595, 842);

            // 创建页面容器
            CDiv container = page.createDivElement();
            container.setPadding(40);

            // 创建标题样式
            CStyle titleStyle = document.createStyle();
            titleStyle
                .setFontSize(24)
                .setFontWeight(CFontWeight.BOLD);

            titleStyle.setMarginBottom(20)
                .setTextAlignment(CTextAlignment.CENTER);

            // 添加标题
            CText title = container.createTextElement("销售发票");
            title.addStyle(titleStyle);

            // 添加发票信息
            CDiv infoSection = container.createDivElement();
            infoSection.setMarginBottom(20);

            CParagraph invoiceNo = infoSection.createParagraphElement("发票编号: INV-2024-001");
            invoiceNo.setFontSize(12);

            CParagraph date = infoSection.createParagraphElement("日期: 2024-01-15");
            date.setFontSize(12);

            // 创建产品表格
            CTable table = container.createTableElement(4);
            table.useAllAvailableWidth();
            table.setMarginBottom(20);

            // 表头
            table.startNewHeaderRow();
            table.createHeaderCell().createTextElement("产品名称");
            table.createHeaderCell().createTextElement("数量");
            table.createHeaderCell().createTextElement("单价");
            table.createHeaderCell().createTextElement("金额");

            // 表格数据
            String[] products = { "产品 A", "产品 B", "产品 C" };
            int[] quantities = { 2, 5, 3 };
            double[] prices = { 100.0, 50.0, 75.0 };

            for (int i = 0; i < products.length; i++) {
                table.startNewRow();
                table.createCell().createTextElement(products[i]);
                table.createCell().createTextElement(String.valueOf(quantities[i]));
                table.createCell().createTextElement("¥" + prices[i]);
                table.createCell().createTextElement("¥" + (quantities[i] * prices[i]));
            }
```

```
            // 计算总计
            double total = 0;
            for (int i = 0; i < products.length; i++) {
                total += quantities[i] * prices[i];
            }

            // 表尾 – 总计
            table.startNewFooterRow();
            CCell totalLabelCell = table.createFooterCell(1, 3); // rowspan, colspan
            totalLabelCell.createTextElement("总计");
            totalLabelCell.setTextAlignment(CTextAlignment.RIGHT);
            totalLabelCell.setFontWeight(CFontWeight.BOLD);

            CCell totalAmountCell = table.createFooterCell();
            totalAmountCell.createTextElement("¥" + total);
            totalAmountCell.setFontWeight(CFontWeight.BOLD);

            // 添加备注
            CParagraph notes = container.createParagraphElement("备注:请在 30 天内付款。");
            notes.setFontSize(10);
            notes.setFontColor(new CColor(100, 100, 100, 255));

            // 保存文档
            document.saveToFile("invoice.pdf");
    }
}
```

创建报告文档

```
public class ReportExample {
    public void generateReport() {
        CDocument document = new CDocument();

            CPage page = document.createPage(595, 842);
            CDiv container = page.createDivElement();
            container.setPadding(50);

            // 标题
            CText title = container.createTextElement("年度报告");
            title.setFontSize(28);
            title.setFontWeight(CFontWeight.BOLD);
            title.setTextAlignment(CTextAlignment.CENTER);

            // 添加章节
            addSection(container, "1. 概述",
                "本报告概述了公司在过去一年的主要成就和发展方向...");

            // 添加图表
            CDiv chartSection = container.createDivElement();
            chartSection.setMarginTop(20);
            chartSection.setMarginBottom(20);

            CText chartTitle = chartSection.createTextElement("2. 销售数据");
            chartTitle.setFontSize(18);
            chartTitle.setFontWeight(CFontWeight.BOLD);

            CImage chart = chartSection.createImageElement("path/to/chart.png");
            chart.setWidth(400);
            chart.setHeight(300);
            chart.setHorizontalAlignment(CHorizontalAlignment.CENTER);

            // 添加关键指标列表
            CText metricsTitle = container.createTextElement("3. 关键指标");
            metricsTitle.setFontSize(18);
            metricsTitle.setFontWeight(CFontWeight.BOLD);

            CList metricsList = container.createListElement();
            metricsList.setListSymbol(CListSymbolType.DISC);
            metricsList.createListItemElement("年度收入增长: 25%");
            metricsList.createListItemElement("客户满意度: 92%");
            metricsList.createListItemElement("市场份额提升: 3%");

            document.saveToFile("report.pdf");

    }

    private void addSection(CDiv container, String heading, String content) {
        CParagraph headingPara = container.createParagraphElement(heading);
        headingPara.setFontSize(18);
        headingPara.setFontWeight(CFontWeight.BOLD);
        headingPara.setMarginTop(20);
        headingPara.setMarginBottom(10);

        CParagraph contentPara = container.createParagraphElement(content);
        contentPara.setFontSize(12);
        contentPara.setTextAlignment(CTextAlignment.JUSTIFIED);
        contentPara.setMultipliedLeading(1.5f);
    }
}
```

3.3.15 最佳实践

资源管理

```
  CDocument document = new CDocument();
  CBorder border = new CBorder(1f);
  CBorderRadius radius = new CBorderRadius(5f)
  // 使用这些资源
  document.saveToFile("output.pdf");
```

样式复用

```
// 创建可重用的样式
CStyle bodyTextStyle = document.createStyle();
bodyTextStyle
    .setFontSize(12)
    .setFontFamily("Arial");

bodyTextStyle.setMarginBottom(10);

CStyle headingStyle = document.createStyle();
headingStyle
    .setFontSize(18)
    .setFontWeight(CFontWeight.BOLD);

headingStyle.setMarginTop(15)
    .setMarginBottom(10);

// 应用到多个元素
paragraph1.addStyle(bodyTextStyle);
paragraph2.addStyle(bodyTextStyle);
heading1.addStyle(headingStyle);
heading2.addStyle(headingStyle);
```

性能优化

```
// 1. 批量创建元素时,考虑预先创建样式
CStyle itemStyle = document.createStyle();
itemStyle.setFontSize(12);
itemStyle.setMarginBottom(5);

for (int i = 0; i < 1000; i++) {
  CText text = div.createTextElement("Item " + i);
  text.addStyle(itemStyle); // 复用样式而不是每次设置
}

//2. 使用流输出而不是先写入内存
try (FileOutputStream fos = new FileOutputStream("large.pdf")) {
  document.saveToFile(fos);
} catch (IOException e) {
  e.printStackTrace();
}
```

错误处理

```
try {
        CDocument document = new CDocument();
        CPage page = document.createPage(595, 842);

        // 验证图片路径
        String imagePath = "path/to/image.png";
        File imageFile = new File(imagePath);
        if (imageFile.exists()) {
            CImage image = page.createDivElement().createImageElement(imagePath);
        } else {
            System.out.println("图片文件不存在: " + imagePath);
        }

        document.saveToFile("output.pdf");

} catch (Exception ex) {
    System.out.println("生成 PDF 时出错: " + ex.getMessage());
    ex.printStackTrace();
}
```

3.3.16 常见问题

Q：如何设置页面方向为横向？

A：创建页面时交换宽度和高度值：

```
// 纵向 A4: 595 x 842
CPage portraitPage = document.createPage(595, 842);

// 横向 A4: 842 x 595
CPage landscapePage = document.createPage(842, 595);
```

Q：如何实现多列布局？

A：使用表格或多个 div 配合宽度设置：

```
CTable layout = page.createTableElement(2);
layout.useAllAvailableWidth();
layout.setBorderCollapse(CBorderCollapseType.COLLAPSE);

// 隐藏边框
CBorder noBorder = new CBorder(0);
layout.setBorder(noBorder);

layout.startNewRow();
CCell leftColumn = layout.createCell();
CCell rightColumn = layout.createCell();

// 在各列中添加内容
leftColumn.createTextElement("左列内容");
rightColumn.createTextElement("右列内容");
```

3.4 邮件转换为 HTML 和 PDF

3.4.1 格式与初始化

1.3.0 版本新增 `CEmailConverter`，支持 EML/MIME 邮件和 Outlook MSG 文件。输入可以是文件路径、`File` 对象、字节数组或输入流；输出可以是 HTML 资源包，或包含邮件正文及可选附件列表的 PDF。

请使用版本匹配的 JAR 和 native 库。运行以下 PDF 示例前，应按第 2.3 节使用有效许可证初始化 SDK。仅解析邮件或导出 HTML 包不需要许可证，但仍须加载 native 库；只使用 HTML 功能的应用可通过 `System.load()` 按绝对路径加载匹配的 native 库。

`CEmailFormat` 提供 `EMAIL_FORMAT_AUTO`、`EMAIL_FORMAT_EML` 和 `EMAIL_FORMAT_MSG`。AUTO 根据 Compound File Binary 文件头识别 MSG，其余输入按 EML 解析，不依赖文件扩展名。二进制 MSG 应直接以文件、字节或流传入，不要先转成 Java 字符串。

3.4.2 邮件文件直接转换为 PDF

```
import CPDFGeneration.CEmailConverter;
import CPDFGeneration.CEmailDiagnostic;
import CPDFGeneration.CEmailResult;

try (CEmailResult result = CEmailConverter.ConvertToPdf("message.eml", "message.pdf")) {
    if (!result.isSuccess()) {
        for (int i = 0; i < result.GetDiagnosticCount(); i++) {
            CEmailDiagnostic diagnostic = result.GetDiagnostic(i);
            System.err.println(diagnostic.code + ": " + diagnostic.detail);
        }
        throw new IllegalStateException("Email conversion failed: " + result.GetStatus(),
                result.GetLastCallbackException());
    }
    System.out.println("Converted format: " + result.GetFormat());
}
```

将 `message.eml` 换为 MSG 路径，即可转换 MSG。输出 PDF 必须尚不存在：路径输出会保留已有文件；失败时会删除本次操作创建的部分文件。无论成功或失败，都应关闭返回的 `CEmailResult`。`EMAIL_SUCCESS` 和 `EMAIL_SUCCESS_WITH_WARNINGS` 都表示成功，`isSuccess()` 会同时接受这两种状态。

PDF 包含选定的邮件正文和启用时的附件列表，不会嵌入附件文件，也不会把附件内容转换成额外页面。PDF 中不保留附件下载链接；需要附件文件时，应导出 HTML 包或读取附件数据。

3.4.3 分别导出 HTML 与生成 PDF

先通过 `ConvertToHtml()` 解析一次，再根据保留的内容导出离线 HTML 包或生成 PDF。下面的示例同时查询主正文来源，并设置自定义页边距。

```
import CPDFGeneration.CConverterProperties;
import CPDFGeneration.CEmailBodySource;
import CPDFGeneration.CEmailConverter;
import CPDFGeneration.CEmailResult;
import CPDFGeneration.CEmailStatus;

CConverterProperties properties = new CConverterProperties();
try {
    properties.setPageMargins(48, 40, 48, 40);
    try (CEmailResult result = CEmailConverter.ConvertToHtml("message.msg")) {
        if (!result.isSuccess()) {
            throw new IllegalStateException("Email parsing failed: " + result.GetStatus());
        }
        CEmailBodySource body = result.GetSelectedBodySource();
        if (body.status.isSuccess()) {
            System.out.println(body.mediaType + " / " + body.origin);
        }
        CEmailStatus exported = result.ExportHtml("mail-package");
        if (!exported.isSuccess()) {
            throw new IllegalStateException("HTML export failed: " + exported);
        }
        CEmailStatus written = result.WritePdf("message.pdf", properties);
        if (!written.isSuccess()) {
            throw new IllegalStateException("PDF write failed: " + written,
                    result.GetLastCallbackException());
```

```
        }
    }
} finally {
    properties.release();
}
```

HTML 包建议使用新的输出目录。`ExportHtml()` 导出 `index.html`、正文资源和启用的附件文件，不覆盖已有输出文件。失败时会清理本次导出创建的文件，并保留调用方原有数据。`GetHtml()` 只返回 UTF-8 HTML 字符串，其中可能包含相对资源路径；需要完整离线资源包时，应使用 `ExportHtml()`。

必须检查每次导出或写入返回的状态。`GetStatus()` 表示最近一次操作，可能替换最初的转换状态。新的 native 输出操作会根据初始 HTML 转换状态重新计算状态和诊断；此前取得的 Java 快照仍然有效。

PDF、授权或写入失败后，已经生成成功的 HTML、资源和附件可以保留，用于再次导出或重试 PDF。输入解析失败则不保证存在 HTML。`GetSelectedBodySource().status` 仅表示正文来源查询的状态，不修改结果对象的操作状态。该接口只标识选定的主正文，不枚举所有显示的 multipart 片段；来源 ID 是结果内部的不透明标识。 返回的 `CEmailBodySource` 包含 `bodyId`、`sourceId`、`mediaType` 和 `origin`；媒体类型为 `text/plain` 或 `text/html`，来源类别为 `original`、`recovered`、`fallback` 或 `generated`。

PDF 属性传 `null` 时，邮件转换默认使用 A4，上下边距为 32 CSS px，左右为 40 CSS px。传入属性对象后，将使用其页面大小、页边距（包括 0）、字体提供者和页眉页脚设置；操作不会修改调用方的属性或 base URI。HTML 预览的内边距不会叠加到 PDF 页边距上。打印配置详见第 3.2.4 和 3.2.5 节。

3.4.4 选项与取消

选项传 `null` 时使用默认值，也可以创建 `CEmailOptions`。转换开始时会复制选项；复制期间不要并发修改同一个对象。

字段	默认值	说明
`flags`	`7`	显示邮件头、显示附件列表并启用附件导出。
`maxInputBytes`	100 MiB	原始输入的最大字节数。
`maxDecodedBytes`	256 MiB	解码内容的字节预算。
`maxHtmlBytes`	64 MiB	生成 HTML 的字节预算。
`cancellationCallback`	`null`	可选 `CEmailCancelCallback`，返回 `true` 请求取消。

`CEmailFlags` 可通过按位 OR 组合：

标志	作用
`EMAIL_INCLUDE_HEADERS`	显示邮件头摘要，默认开启。
`EMAIL_INCLUDE_ATTACHMENT_LIST`	显示附件列表，默认开启。
`EMAIL_EXPORT_ATTACHMENTS`	随 HTML 包导出可用的附件文件，默认开启。
`EMAIL_INLINE_RESOURCES`	在生成的 HTML 中内联正文资源。
`EMAIL_RECOVER_EML`	对不规范 EML 启用有限恢复；应通过诊断检查降级或跳过的内容。

标志	作用
EMAIL_INCLUDE_BCC	显示默认隐藏的 Bcc。

```
import CPDFGeneration.CEmailConverter;
import CPDFGeneration.CEmailFlags;
import CPDFGeneration.CEmailFormat;
import CPDFGeneration.CEmailOptions;
import CPDFGeneration.CEmailResult;
import java.util.concurrent.atomic.AtomicBoolean;

CEmailOptions options = new CEmailOptions();
options.flags |= CEmailFlags.EMAIL_INLINE_RESOURCES;
options.maxInputBytes = 20L * 1024 * 1024;
AtomicBoolean cancelled = new AtomicBoolean(false);
options.cancellationCallback = cancelled::get;
try (CEmailResult result = CEmailConverter.ConvertToHtml(
        "message.eml", CEmailFormat.EMAIL_FORMAT_AUTO, options)) {
    if (!result.isSuccess()) {
        throw new IllegalStateException("Email parsing failed: " + result.GetStatus(),
                result.GetLastCallbackException());
    }
    System.out.println("HTML characters: " + result.GetHtml().length());
}
```

字节预算必须为非负数；0 表示真实的零上限，不代表无限。负预算在传给转换器时抛出 `IllegalArgumentException`。未知标志位返回 `EMAIL_ERROR_INVALID_PARAMETER`。即使没有超过配置的字节预算，仍可能触发解析器内部的结构限制。

取消采用协作式检查，发生在阶段之间和回调 I/O 中，不会中断正在执行的解析或排版操作，也不提供严格的超时保证。回调在当前调用 Java 线程上同步执行，应快速返回。其他线程可将示例中的 `cancelled` 标志设为 `true` 来请求取消。

结果对象会持有回调，后续 `ExportHtml()` 和 `WritePdf()` 仍使用该回调，直到结果关闭。修改原选项对象不会替换已保留的回调。回调异常会被视为取消，并保存在 `GetLastCallbackException()` 中。回调内不要重入 SDK、关闭正在使用的结果、释放其属性，或同步等待另一项 PDF 任务。

3.4.5 流与内存输入

使用输入流时，应提供从当前位置开始要读取的邮件字节数，不要把 `InputStream.available()` 当成邮件长度。文件输入可使用 `Files.size()`，并确保读取期间输入不发生变化。

```
import CPDFGeneration.CEmailConverter;
import CPDFGeneration.CEmailResult;
import java.io.InputStream;
import java.io.OutputStream;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;

Path input = Paths.get("message.eml");
Path output = Paths.get("message-stream.pdf");
long length = Files.size(input);
try (InputStream source = Files.newInputStream(input);
     OutputStream destination = Files.newOutputStream(output,
             StandardOpenOption.CREATE_NEW, StandardOpenOption.WRITE);
     CEmailResult result = CEmailConverter.ConvertToPdf(source, length, destination)) {
    if (!result.isSuccess()) {
        throw new IllegalStateException("Stream conversion failed: " + result.GetStatus(),
                result.GetLastCallbackException());
```

```
    }
}
```

短读会循环补足。提前结束会返回 `EMAIL_ERROR_READ`，并在结果中保留原始 `EOFException`。SDK 不关闭或 flush 调用方的流；上面的 try-with-resources
由调用方负责关闭。失败或取消后，应丢弃部分输出，包括调用方为输出流自行创建的文件。

完全使用内存进行转换：

```
import CPDFGeneration.CEmailConverter;
import CPDFGeneration.CEmailResult;
import java.io.ByteArrayOutputStream;
import java.nio.file.Files;
import java.nio.file.Paths;

byte[] email = Files.readAllBytes(Paths.get("message.msg"));
ByteArrayOutputStream output = new ByteArrayOutputStream();
try (CEmailResult result = CEmailConverter.ConvertToPdf(email, output)) {
    if (!result.isSuccess()) {
        throw new IllegalStateException("Memory conversion failed: " + result.GetStatus(),
                result.GetLastCallbackException());
    }
    byte[] pdf = output.toByteArray();
    System.out.println("PDF bytes: " + pdf.length);
}
```

内存输入和输出会占用 Java
堆空间。较大的邮件建议使用文件或流输入，只有需要附件内容时才复制附件字节。

下列重载组都提供简化方法和带显式格式、选项的完整方法；PDF 完整方法还接受 `CConverterProperties`：

操作	输入	输出	完整方法的附加参数
`ConvertToHtml`	`String` 路径或 `File`	`CEmailResult`	`CEmailFormat`、`CEmailOptions`
`ConvertToHtml`	`byte[]`	`CEmailResult`	`CEmailFormat`、`CEmailOptions`
`ConvertToHtml`	`InputStream`、`long length`	`CEmailResult`	`CEmailFormat`、`CEmailOptions`
`ConvertToPdf`	`String` 路径	`String` PDF 路径	`CEmailFormat`、`CEmailOptions`、`CConverterProperties`
`ConvertToPdf`	`File`	`File` PDF 输出	`CEmailFormat`、`CEmailOptions`、`CConverterProperties`
`ConvertToPdf`	`byte[]`	`OutputStream`	`CEmailFormat`、`CEmailOptions`、`CConverterProperties`
`ConvertToPdf`	`InputStream`、`long length`	`OutputStream`	`CEmailFormat`、`CEmailOptions`、`CConverterProperties`

简化方法使用 AUTO 格式及默认选项；PDF
简化方法还使用邮件的默认页面属性。所有转换方法均返回需要关闭的自有 `CEmailResult`。

3.4.6 结果数据与状态码

结果方法	用途
`GetStatus()` / `isSuccess()`	最近一次转换或 native 输出操作的状态。
`GetFormat()`	解析器报告的输入格式。
`GetPdfError()`	PDF 或授权失败时的底层 `CHtmlConverter.CErrorCode`。
`GetHtml()`	HTML 字符串；未生成 HTML 时为空字符串。
`GetResourceCount()` / `GetResource(index)`	正文资源数量及元数据、字节快照。
`GetAttachmentCount()` / `GetAttachment(index)`	附件数量及元数据、字节快照。
`GetDiagnosticCount()` / `GetDiagnostic(index)`	错误或警告诊断，包括代码、来源 ID、详情和降级处理。
`GetSelectedBodySource()`	主正文来源的查询状态、ID、媒体类型和来源类别。
`GetLastCallbackException()`	保留的 Java 流或取消回调异常；没有时为 `null`。
`ExportHtml(String / File)`	导出离线 HTML 包。
`WritePdf(String / File / OutputStream [, properties])`	从保留的 HTML 生成或重试 PDF。

`GetResource()` 返回 `CEmailResource`，`GetAttachment()` 返回 `CEmailAttachment`，`GetDiagnostic()` 返回 `CEmailDiagnostic`。资源和附件提供媒体类型、相对路径、字节数等元数据。附件还提供 `id`、`originalName`、`displayName`、`hasData`、`hidden`、`referencedByBody` 和 `nestedMessage`。只有元数据的附件满足 `hasData == false` 和 `getData() == null`，与合法的零字节附件不同。

集合索引从 0 开始，越界时抛出 `IndexOutOfBoundsException`。元数据及字节快照归 Java 所有，结果关闭后仍然有效。每次 `getData()` 还会返回一个新的独立字节数组，因此较大的内容会额外占用堆空间。

CEmailStatus	数值	含义
`EMAIL_SUCCESS`	`0`	无警告的成功。
`EMAIL_SUCCESS_WITH_WARNINGS`	`1`	成功，但有警告或降级内容。
`EMAIL_ERROR_INVALID_PARAMETER`	`-1`	参数、格式或选项无效。
`EMAIL_ERROR_READ`	`-2`	读取输入失败。
`EMAIL_ERROR_PARSE`	`-3`	邮件解析失败。
`EMAIL_ERROR_HTML`	`-4`	HTML 生成失败。
`EMAIL_ERROR_WRITE`	`-5`	输出写入失败。
`EMAIL_ERROR_PDF`	`-6`	PDF 渲染失败。

CEmailStatus	数值	含义
EMAIL_ERROR_LICENSE	-7	PDF 授权检查失败。
EMAIL_ERROR_LIMIT	-8	超出字节预算或内部结构限制。
EMAIL_ERROR_CANCELLED	-9	已请求取消。
EMAIL_ERROR_OUT_OF_MEMORY	-10	native 内存分配失败。
EMAIL_ERROR_INTERNAL	-11	内部错误或未知的 native 状态。

一般转换错误通过状态返回。负预算、路径含 NUL 等 Java 参数验证错误可能抛出 `IllegalArgumentException`；Java 或 JNI 内存分配失败仍可能抛出 `OutOfMemoryError`。流与取消回调异常保存在 `GetLastCallbackException()` 中，下一次 native 输出操作会替换该值。Java 参数验证失败不会改变前一次操作的状态。

3.4.7 生命周期与并发

- 每个 `CEmailResult` 都应使用 try-with-resources。`close()` 和 `release()` 可重复调用；关闭后的 native 查询或输出会抛出 `IllegalStateException`，仍可调用 `isClosed()`。
- 转换期间保持属性对象及其字体提供者有效且不被修改。邮件操作与属性 `release()` 使用同一锁；已释放的属性不能再次传给邮件 API。
- 独立结果可以并发解析或导出；同一个 Java 结果的操作与关闭会串行执行。
- 邮件 PDF 渲染在内部串行化。不要同时通过旧 HTML API 渲染，也不要在 PDF 渲染期间修改全局授权或字体状态。
- 遵守第 3.4.4 节的回调限制，避免重入或等待正在执行的转换。
- Java 路径和元数据采用 UTF-8。Linux JDK 8 的文件系统 locale 也需使用 UTF-8，才能正确处理非 ASCII 路径。

3.4.8 邮件示例

从发布包的 `samples` 目录运行：

```
cd EmailConvertTest
bash RunTest.sh
```

示例直接读取 `samples/TestFiles/email` 中固定的 EML/MSG 文件，将 HTML 包、带页眉页脚的 PDF、嵌套邮件 PDF 和流转换 PDF 输出到新的 `samples/out/email-example-*` 目录。Linux 脚本会从 PATH 对应的 Java 安装中定位并预加载 `libjsig.so`，保持 JVM 信号处理器兼容性。在线授权缓存的后台刷新仍默认开启。

`PrintOptionsTest` 演示页边距和页眉页脚配置；`FamilyHeaderFooterTest` 使用 `TestFiles/family_html/family_all_pages.html` 演示封面跳页及页码设置。

4 技术支持

4.1 报告问题

感谢您对 ComPDF Generation SDK 的关注，这是一款易用且功能强大的开发解决方案，可以将高质量的 PDF 生成功能集成到您的应用程序中。如果您在使用 ComPDF Generation SDK
时遇到任何技术问题，请将问题报告提交给 ComPDF 团队。更多的信息如下，将有助于我们解决您的问题：

- ComPDF Generation SDK 产品和版本。
- 你的操作系统和 IDE 版本。
- 问题的详细描述。
- 任何其他相关信息，例如错误截图。

4.2 联系信息

网站链接：

- 英文主页：https://www.compdf.com
- 中文主页：https://www.compdf.com/zh-cn
- 开发者文档：https://www.compdf.com/guides/generation-sdk/java/overview
- API 参考文档：https://developers.compdf.com/guides/generation-sdk/java/api-reference/

联系 **ComPDF**：

- 联系销售：https://www.compdf.com/contact-sales
- 技术问题反馈至：https://www.compdf.com/support
- 电子邮件（直接联系）：support@compdf.com

谢谢，

ComPDF 团队